



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

к лабораторной работе №7

По курсу: «Экономика программной инженерии»

*Тема: «Оценка параметров программного проекта с использованием методов
функциональных точек и модели СОСОМО II»*

Вариант №1

Студентка ИУ7-84Б
Цховребова Я.Р.

Студент ИУ7-84Б
Гаврилов В.А.

Студент ИУ7-84Б
Бугаков И.С.

Преподаватель
Барышникова М.Ю.
Силантьева А.В.

2026г.

1. Цель работы

Целью лабораторной работы является продолжение знакомства с существующими методиками предварительной оценки параметров программного проекта и практическая оценка затрат по модели COSOMO II.

2. Методика функциональных точек

Данный метод и измеряет **функциональный размер** ПО (что система делает), а не объём кода. Это позволяет оценить трудоёмкость разработки **на ранних этапах** (по требованиям), независимо от языка и технологии. Позволяет заменить устаревшую метрику «строки кода» (LOC), которая зависит от стиля программирования и не применима до написания кода. FP даёт более стабильную и предсказуемую оценку.

3. COSOMO II

COSOMO II — это обновлённая версия классической модели COSOMO для оценки трудоёмкости и стоимости разработки ПО. Она учитывает современные методы разработки (быстрая разработка, повторное использование, инструментарий) и позволяет проводить оценку на разных этапах проекта — от предварительного (ещё нет архитектуры) до детального (архитектура уже готова).

Включает три варианта:

- модель композиции приложения;
- модель ранней разработки архитектуры;
- постархитектурная модель.

4. Проект

Компания получила заказ на разработку автоматизированной информационной системы оплаты штрафов ГИБДД. Оплата штрафов возможна через веб-интерфейс веб-портала и через приложение для мобильного телефона.

Роли пользователей

В системе предусмотрено два вида пользователей:

- Пользователь (User),

- Администратор (Administrator).

Пользователь может просматривать и оплачивать штрафы. Администратору доступен просмотр всех выплат, просмотр данных пользователей, их редактирование и создание новых пользователей.

Все записи типа «Пользователь» в системе имеют следующие поля:

- *id*,
- *логин*,
- *пароль*,
- *тип*,
- *регистрационный номер водительского удостоверения*,
- *номер банковской карты*.

Модули

Информационная система состоит из следующих модулей:

- приложение для мобильного телефона;
- веб-портал;
- модуль регистрации и авторизации;
- модуль обмена данными с системой ГИБДД;
- проведения платежных транзакций.

Приложение для мобильного телефона

Содержит:

- страницу регистрации, где пользователь заполняет следующие поля: логин, пароль, номер водительского удостоверения, номер банковской карты;
- страницу просмотра штрафов, на которой отображаются неоплаченные штрафы, и есть кнопка “Оплатить” для каждого штрафа. После нажатия кнопки пользователю приходит ответ о положительном или отрицательном результате выполнения операции.

Веб-портал

Обеспечивает те же функциональные возможности для пользователя в дополнение к этому он имеет панель администратора.

Модель регистрации и авторизации

Позволяет добавлять пользователей в базу данных.

Модуль обмена данными с системой ГИБДД

Предназначен для получения списка штрафов, а также оповещения ГИБДД об оплате штрафа. При отправке сообщения с номером водительского удостоверения информационной системе ГИБДД модуль обмена данными получает список штрафов.

При этом каждый штраф описывается следующими полями: номер постановления, дата постановления, имя, фамилия, отчество нарушителя, сумма штрафа.

На сообщение об оплате система ГИБДД присылает подтверждение об удалении штрафа из списка неоплаченных или отказ.

Система не хранит штрафы в своей базе данных, а получает их из системы ГИБДД при каждом запросе.

Модуль проведения платежных транзакций

Модуль по защищенному протоколу отправляет платёжной системе запрос с указанием номера карты пользователя, номера счета ГИБДД и суммы на выполнение проведения оплаты.

Система отсылает положительный или отрицательный ответ о результате выполнения. В отличие от штрафов все операции по оплате сохраняются в базу данных.

Характеристики команды, продукта и проекта

Существует 14 оценок степени влияния общих системных характеристик из метода функциональных точек (IFPUG). Каждая характеристика оценивается по шкале от 0 («нет влияния») до 5 («сильное влияние») на основе анализа требований:

- обмен данными — 5 – система активно обменивается данными с ГИБДД и платёжной системой;
- распределенная обработка — 5 – есть веб-портал и мобильное приложение;

- производительность — 3 – средние требования к скорости ответа;
- эксплуатационные ограничения по аппаратным ресурсам — 0;
- транзакционная нагрузка — 3 – умеренное число платежей и запросов штрафов;
- интенсивность взаимодействия с пользователем (оперативный ввод) — 2 – операции не требуют постоянного ввода;
- эргономические характеристики, влияющие на эффективность работы конечных пользователей — 0;
- оперативное обновление — 4;
- сложность обработки — 4 – логика платежей, проверок и взаимодействий с внешними системами;
- повторное использование — 3;
- легкость инсталляции — 0;
- легкость эксплуатации/администрирования — 3;
- портируемость — 5 – требуется работа на веб и мобильных платформах;
- гибкость — 0.

При разработке ПО:

- 30 % кода будет написано на SQL,
- 10 % — на JavaScript, 6
- 0 % — на Java.

К разработке проекта планируется привлечь довольно слаженную команду высокопрофессиональных разработчиков, у которых, однако, практически отсутствует опыт в разработке систем подобного типа. При этом заказчик настаивает на довольно строгом процессе с периодической демонстрацией рабочих продуктов, соответствующих этапам жизненного цикла. Учитывая новизну проекта для команды, на этапе его подготовки был осуществлен относительно детальный анализ рисков, свойственных архитектуре разрабатываемой системы. Организация находится чуть выше второго уровня зрелости процессов разработки.

Надежность и уровень сложности (RCPX) разрабатываемой системы оцениваются как очень высокие, проект не предусматривает специальных усилий на повторное использование компонентов (RUSE). Возможности персонала (PERS) можно охарактеризовать как очень высокие, однако опыт членов команды в данной сфере (PREX) является скорее низким. Сложность платформы (PDIF)

средняя. Разработка предусматривает интенсивное использование инструментальных средств поддержки (FCIL). Учитывая новизну проекта для команды и высокие требования по надежности. Заказчик не настаивает на жестком графике (SCED).

5. Лабораторная работа

В данной работе рассматриваются модель композиции приложения и модель ранней разработки архитектуры.

Факторы затрат для модели ранней разработки архитектуры COCOMO II:

- новизна проекта (PREC) — почти полное отсутствие прецедентов, в значительной мере непредсказуемый проект, увеличивает трудоемкость;
- гибкость процесса разработки (FLEX) — точный, строгий процесс разработки, увеличивает трудоемкость;
- разрешение рисков в архитектуре системы (RESL) — в целом (75%), немного снижает трудоемкость;
- сплоченность команды (TEAM) — взаимодействие как в едином целом, снижает трудоемкость;
- уровень развития процесса разработки (PMAT) — уровень 2 (уровень по CMM), повторяемый процесс (бывают базовые практики, но не очень зрелый).

Расчет по методу функциональных точек

FTR — количество связанных с каждым функциональным типом файлов типа ссылок, сколько IFL/EIF затрагивает процесс (количество таблиц, которые использует действие).

DET — количество связанных с каждым функциональным типом элементарных данных (количество полей на экране / отчете).

RET — количество типов элементов записей (количество разных “подтаблиц” внутри одной логической таблицы).

EI (внешний ввод) — элементарный процесс, перемещающий данные из внешней среды в приложение.

EO (внешний вывод) — элементарный процесс, перемещающий данные, вычисленные в приложении, во внешнюю среду.

EQ (внешний запрос) — элементарный процесс, состоящий из комбинации «запрос/ответ», не связанный с вычислением производных данных или обновлением внутренних логических файлов (базы данных).

ILF (внутренний логический файл) — данные, которые система сама хранит и меняет.

EIF (внешний интерфейсный файл) — данные, которые система читает, но сама не хранит.

В нашем приложении используются два внутренних файла: пользователи, транзакции, и один внешний – штрафы из системы ГИБДД.

Вычисление EI

- Регистрация
 - FTR = 1 (один внутренний логический файл);
 - DET = 4 (логин, пароль, номер водительского удостоверения и номер банковской карты).
- Оплата штрафа
 - FTR = 1 (один внутренний логический файл);
 - DET = 1 (дата оплаты).
- Добавление пользователя администратором;
 - FTR = 1 (один внутренний логический файл);
 - DET = 5 (логин, пароль, тип, номер карты, номер удостоверения).
- Редактирование пользователя администратором;
 - FTR = 1 (один внутренний логический файл);
 - DET = 5 (логин, пароль, тип, номер карты, номер удостоверения).

Уровень сложности – низкий.

Вычисление EO

- Вывод списка штрафов
 - FTR = 1 (один внутренний логический файл);
 - DET = 4 (номер, ФИО, дата, сумма).
- Вывод сообщения о статусе оплаты
 - FTR = 1 (один внутренний логический файл);

- DET = 1 (статус оплаты).
- Вывод информации о пользователе
 - FTR = 1 (один внутренний логический файл);
 - DET = 6 (все поля из базы).

Уровень сложности – низкий.

Вычисление EQ

- Запрос на авторизацию
 - FTR = 1 (один внутренний логический файл);
 - DET = 3 (логин, пароль, флажок).
- Запрос на вывод списка
 - FTR = 1 (один внутренний логический файл);
 - DET = 4 (номер, ФИО, дата, сумма).

Уровень сложности – низкий.

Вычисление ILF

- RET = 3 (элементы записи);
- DET = 6 (элементы данных).

Уровень сложности – низкий.

Вычисление EIF

- Веб
 - RET = 3 (элементы записи);
 - DET = 10 (элементы данных).
- Мобильное приложение
 - RET = 3 (элементы записи);
 - DET = 5 (элементы данных).

Уровень сложности — низкий.

Результаты расчетов

Метод функциональных точек

- Нормированное количество функциональных точек: 47.94

- Количество функциональных точек: 47
- Количество строк исходного кода: 3590

Метрика	Величина	Внешний вид	Количество	Сложность	Итого	Процент использования языков
Передачи данных	5					
Распределенная обработка данных	5	Внешние вводы (EI)	4	Низкий (3)	12	ASM 0
Производительность	3	Внешние выходы (EO)	3	Низкий (4)	12	C 0
Эксплуатационные ограничения	0	Внешние запросы (EQ)	2	Низкий (3)	6	Cobol 0
Частота транзакций	3	Внутренние логические файлы (ILF)	1	Низкий (7)	7	Fortran 0
Оперативный ввод данных	2	Внешние интерфейсные файлы (EIF)	2	Низкий (6)	10	Pascal 0
Эффективность работы конечных пользователей	0					
Оперативное обновление	4					Java 60
Сложность обработки	4					C# 0
Повторное использование	3					Ada 95 0
Легкость установки	0					SQL 30
Легкость эксплуатации	3					Visual C++ 0
Портируемость	5					Delphi Pascal 0
Простота изменений	0					Perl 0
						JavaScript 10
Итого					47	

Оценка по модели COCOMO II

- $p = 1.1964$

Факторы показателя степени модели

Новизна проекта (PREC): Полное отсутствие прецедентов, полностью непредсказуемый проект

Гибкость процесса разработки (FLEX): Точный, строгий процесс разработки

Анализ архитектуры системы и рисков (RESL): В целом (75 %)

Сплоченность команды (TEAM): Высокая согласованность

Уровень развития процесса разработки (PMAT): Уровень 2 CMM

Р равно: 1.2088

Модель ранней разработки архитектуры

PERS: Очень высокий

RCPX: Очень высокий

RUSE: Низкий

PDIF: Номинальный

PREX: Низкий

FDIL: Очень высокий

SCED: Низкий

Модель композиции приложения

Экраны формы: Простые 3, Средние 1, Сложные 0

Отчеты: Простые 2, Средние 1, Сложные 0

Модули на языках 3 поколения: 3

%RUSE: 0

Опытность команды/разработчика: Низкая

Трудозатраты, чел/мес: 3

Время, мес: 3

Бюджет: 300000.0

Композиция приложения

Формы:

- страница регистрации — простая (обращение к БД);
- страница авторизации — простая (обращение к БД);
- страница штрафов — средняя (обращение к БД ГИБДД);
- страница информации о пользователе — простая (запрос к БД).

Отчеты:

- отчет о пользователях — простой (обращение к БД);
- отчет о штрафах — средний (обращение к БД ГИБДД);
- отчет о платежах — простой (обращение к БД).

Итого:

- простые формы — 3;
- средние формы — 1;
- простые отчеты — 2;
- средние отчеты — 1;
- модули на ЯП третьего поколения — 3;
- повторное использование — 0 %;
- опыт команды — очень низкий;

Модель ранней разработки архитектуры

- PERS (возможности персонала) — очень высокий;
- RCPX (надежность и уровень сложности разрабатываемой системы) — очень высокий;
- RUSE (повторное использование компонентов) — низкий;
- PDIF (сложность платформы разработки) — номинальный;
- PREX (опыт персонал) — низкий;
- FCIL (средства поддержки) — очень высокий;
- SCED (график работ) — низкий;
- KSLOC = 3.6 (из метода функциональных точек).

Выводы

Методика COSOMO II, как и ее первая версия, позволяет досочно быстро оценить длительность и трудозатраты проекта, основываясь на субъективных данных. В условиях отсутствия объективной информации о предполагаемых трудозатратах особенно важно правильно спрогнозировать характеристики проекта.

Методика функциональных точек позволяет оценить размер программного продукта на этапе его проектирования. С помощью этого подхода можно применять методики COSOMO, которым необходимо знание о размере продукта.

Хотя методика функциональных точек является неточной (как и любая методика прогнозирования), этих результатов достаточно для общего понимания объемов работ и получения приблизительных оценок параметров проекта.

В ходе выполнения лабораторной работы было выяснено, что модель композиции приложения дает менее оптимистичный прогноз, по сравнению с моделью ранней архитектуры приложения.